Sample Data

PS-05 - RCA Drafter | 6 sample incidents with input data and expected RCA output

The following incidents cover common production failure patterns across different services. Use these to verify the RCA pipeline, demonstrate the system during the demo, and as regression tests after any code change.

Sample Incident 1 — Database Connection Pool Exhaustion

Severity: P1	Service: payment-api	Duration: 28 minutes

INPUT — Incident Timeline

```
10:00 - v2.4.1 deployed to production (payment-api) 10:03 - Error rate climbs from 0.1% to 12% 10:05 - payment-api pods showing high latency (p99 > 8s) 10:07 - PagerDuty alert fired: payment-api error rate > 5% 10:12 - On-call engineer acknowledged alert 10:18 - DB team alerted, begins investigation 10:28 - Rollback to v2.4.0 initiated 10:31 - Error rate drops to 0.08%, incident resolved
```

INPUT — Error Logs

```
ERROR 2024-01-15 10:03:12 payment-api - Connection pool exhausted ERROR 2024-01-15 10:03:13 payment-api - Timeout after 5000ms FATAL 2024-01-15 10:03:15 db-primary - Too many connections (151/151) ERROR 2024-01-15 10:03:18 payment-api - Cannot get connection, pool error WARN 2024-01-15 10:04:01 payment-api - Retrying connection (attempt 3/3) ERROR 2024-01-15 10:04:03 payment-api - All retry attempts failed
```

INPUT — Git Diff

```
diff --git a/config/database.properties b/config/database.properties --- a/config/database.properties +++ b/config/database.properties @@ -10,4 +10,4 @@ -maxConnections=10 +maxConnections=150 idleTimeout=30000 connectionTimeout=5000
```

EXPECTED OUTPUT — RCA Summary

Field	Expected Value
Root Cause	maxConnections raised from 10 to 150, exceeding DB server hard limit of 151
Severity	P1
Contributing Factor 1	No alerting on DB connection pool utilisation before saturation
Contributing Factor 2	Config change shipped without load testing
Key Action Item	Revert maxConnections to 10; add pool utilisation alert at 80%

Sample Incident 2 — Auth Service Memory Leak — JWT Cache Unbounded

Severity: P2	Service: auth-service	Duration: 45 minutes

INPUT — Incident Timeline

```
14:00 - v3.1.2 deployed to auth-service 14:20 - Memory usage climbs steadily: 60% then 85% 14:35 - Login latency increases to p99 > 3s 14:40 - OOM kill on auth-service pod-2 14:43 - Auto-scaling triggers, new pod starts 14:45 - New pod also shows climbing memory usage 14:55 - Rollback to v3.1.1, memory stabilises at 42%
```

INPUT — Error Logs

```
WARN 2024-01-16 14:20:03 auth-service - Memory usage at 72% WARN 2024-01-16 14:30:11 auth-service - Memory usage at 84% ERROR 2024-01-16 14:40:02 auth-service - OOMKilled: container exceeded memory limit INFO 2024-01-16 14:43:01 k8s - New pod auth-service-7d9f started WARN 2024-01-16 14:46:00 auth-service - Memory at 68% (new pod, still climbing) ERROR 2024-01-16 14:52:11 auth-service - JWT cache size: 847,293 entries
```

INPUT — Git Diff

```
diff --git a/src/auth/jwt_cache.py b/src/auth/jwt_cache.py --- a/src/auth/jwt_cache.py +++ b/src/auth/jwt_cache.py @@ -18,6 +18,5 @@ def store_token(self, token, user_id): - if len(self.cache) > self.MAX_SIZE: - self._evict_oldest() self.cache[token] = {"user_id": user_id, "ts": time.time()}
```

EXPECTED OUTPUT — RCA Summary

Field	Expected Value
Root Cause	JWT cache eviction logic removed in v3.1.2, causing unbounded memory growth
Severity	P2
Contributing Factor 1	No memory-based cache size cap as a secondary safeguard
Contributing Factor 2	Memory alert threshold (85%) set too close to OOM limit
Key Action Item	Restore eviction logic; add cache size metric and alert at 500k entries

Sample Incident 3 — CDN Cache Bypass — Missing Cache-Control Header

Severity: P2	Service: cdn-edge / api-gateway	Duration: 1 hour 12 minutes

INPUT — Incident Timeline

```
09:00 - Marketing campaign goes live, traffic 10x normal 09:04 - api-gateway CPU spikes to 94% 09:08 - Response times degrade: p50 > 2s, p99 > 12s 09:15 - CDN team notices cache hit rate dropped from 87% to 3% 09:22 - Engineers identify missing Cache-Control header in recent deploy 09:35 - Hot-fix deployed restoring Cache-Control: public, max-age=300 10:12 - Cache hit rate recovers to 89%, latency normalises
```

INPUT — Error Logs

```
WARN 2024-02-10 09:04:11 api-gateway - CPU utilisation 94% ERROR 2024-02-10 09:05:02 api-gateway - Request queue depth: 4,821 ERROR 2024-02-10 09:06:18 cdn-edge - Cache MISS ratio: 97% (threshold 20%) WARN 2024-02-10 09:08:44 api-gateway - p99 latency 12,340ms ERROR 2024-02-10 09:09:01 api-gateway - Circuit breaker OPEN on /api/products INFO 2024-02-10 09:22:03 cdn-edge - Cache-Control header absent on all responses
```

INPUT — Git Diff

```
diff --git a/src/middleware/response.py b/src/middleware/response.py --- a/src/middleware/response.py +++ b/src/middleware/response.py @@ -34,5 +34,4 @@ def add_response_headers(response): - response.headers["Cache-Control"] = "public, max-age=300" - response.headers["Vary"] = "Accept-Encoding" response.headers["X-Request-ID"] = generate_id()
```

EXPECTED OUTPUT — RCA Summary

Field	Expected Value
Root Cause	Cache-Control header removed from response middleware, causing full CDN cache bypass
Severity	P2
Contributing Factor 1	No CDN cache-hit rate alert; miss rate reached 97% undetected for 11 min
Contributing Factor 2	High-traffic campaign launched same day as middleware change
Key Action Item	Restore Cache-Control header; add CDN miss-rate alert at >25%

Sample Incident 4 — Deploy Pipeline Broken — Smoke Test Skipped

Severity: P3	Service: deploy-pipeline / notification-worker	Duration: 52 minutes

INPUT — Incident Timeline

```
16:00 - notification-worker v1.8.0 deployed via CI/CD pipeline 16:02 - Deployment marked green by pipeline 16:10 - Support team reports no emails being sent since 16:00 16:18 - Engineer checks notification-worker logs, finds SMTP errors 16:25 - Root cause identified: SMTP_PORT env var missing in new deploy 16:40 - Env var added to deployment config, redeployed 16:52 - Email delivery confirmed restored, backlog processing started
```

INPUT — Error Logs

```
INFO 2024-03-05 16:00:14 deploy-pipeline - notification-worker v1.8.0 deployed OK ERROR 2024-03-05 16:00:31 notification-worker - SMTP connect failed: missing host/port ERROR 2024-03-05 16:00:32 notification-worker - Cannot read env SMTP_PORT: undefined WARN 2024-03-05 16:00:45 notification-worker - Retrying SMTP connection (1/3) ERROR 2024-03-05 16:00:59 notification-worker - All SMTP retries exhausted, dropping job ERROR 2024-03-05 16:10:02 notification-worker - Dead letter queue depth: 1,240 messages
```

INPUT — Git Diff

```
diff --git a/.github/workflows/deploy.yml b/.github/workflows/deploy.yml --- a/.github/workflows/deploy.yml +++ b/.github/workflows/deploy.yml @@ -44,8 +44,6 @@ - name: Deploy notification-worker run: kubectl apply -f k8s/notification-worker.yaml - env: - SMTP_PORT: ${{ secrets.SMTP_PORT }} - SMTP_HOST: ${{ secrets.SMTP_HOST }} - name: Run smoke tests - run: pytest tests/smoke/test_email.py + run: echo "smoke tests skipped"
```

EXPECTED OUTPUT — RCA Summary

Field	Expected Value
Root Cause	SMTP_PORT and SMTP_HOST env vars removed from deploy manifest; smoke test disabled
Severity	P3
Contributing Factor 1	Smoke test that would have caught missing env vars was bypassed
Contributing Factor 2	No health check verifying SMTP connectivity on pod startup
Key Action Item	Restore env vars and smoke tests; add SMTP connectivity readiness probe

Sample Incident 5 — Infinite Retry Loop — Exponential Backoff Removed

Severity: P1	Service: order-service / db-replica	Duration: 38 minutes

INPUT — Incident Timeline

```
02:14 - Scheduled batch job starts order reconciliation 02:15 - db-replica briefly unreachable (routine failover, ~8s) 02:15 - order-service starts retrying DB calls 02:16 - Retry storm begins: 12,000 requests/sec to db-replica 02:18 - db-replica CPU hits 100%, legitimate queries time out 02:22 - order-service pods OOMKilled by retry payload buildup 02:40 - order-service manually restarted, throttling added, db recovers 02:52 - All services stable, batch job re-queued
```

INPUT — Error Logs

```
INFO 2024-04-02 02:14:01 order-service - Batch reconciliation started WARN 2024-04-02 02:15:03 order-service - DB connection lost, retrying ERROR 2024-04-02 02:15:04 order-service - Retry attempt 1 failed ERROR 2024-04-02 02:15:04 order-service - Retry attempt 2 failed ERROR 2024-04-02 02:15:05 order-service - Retry attempt 847 failed FATAL 2024-04-02 02:16:12 db-replica - Max connections reached (500/500) ERROR 2024-04-02 02:22:04 order-service - OOMKilled: heap overflow from retry queue
```

INPUT — Git Diff

```
diff --git a/src/db/retry.py b/src/db/retry.py --- a/src/db/retry.py +++ b/src/db/retry.py @@ -12,9 +12,6 @@ def retry_with_backoff(fn, max_attempts=10): - delay = 0.5 for attempt in range(max_attempts): try: return fn() except DBError: - time.sleep(delay) - delay = min(delay * 2, 30) + continue
```

EXPECTED OUTPUT — RCA Summary

Field	Expected Value
Root Cause	Exponential backoff removed from retry logic; infinite tight-loop on DB failure
Severity	P1
Contributing Factor 1	No maximum retry count enforced; loop ran indefinitely
Contributing Factor 2	DB failover window (8s) normal, but retry storm turned it into a cascade
Key Action Item	Restore backoff with jitter; add max_attempts cap and circuit breaker

Sample Incident 6 — Search Service Down — Index Mapping Conflict

Severity: P2	Service: search-service / elasticsearch	Duration: 1 hour 5 minutes

INPUT — Incident Timeline

```
11:30 - search-service v2.0.0 deployed (new product search feature) 11:31 - All search requests begin returning HTTP 500 11:33 - Users report search bar broken on all product pages 11:40 - Engineer inspects logs, finds Elasticsearch mapping exception 11:48 - New "tags" field in code conflicts with existing keyword mapping 12:00 - Index migration script run to update mapping 12:15 - Migration fails on 2.3M existing documents 12:35 - Rollback to v1.9.4, reindex started in background
```

INPUT — Error Logs

```
ERROR 2024-05-12 11:31:02 search-service - ES error: mapper_parsing_exception ERROR 2024-05-12 11:31:03 search-service - field [tags] of type [text] cannot be changed to [keyword] ERROR 2024-05-12 11:31:04 search-service - Indexing failed for doc_id: prod_00142 WARN 2024-05-12 11:31:10 search-service - Bulk index error rate: 100% ERROR 2024-05-12 12:15:08 search-service - Reindex failed: shard timeout on products_v1 (2.3M docs) INFO 2024-05-12 12:35:00 search-service - Rollback to v1.9.4 complete
```

INPUT — Git Diff

```
diff --git a/src/search/mappings.py b/src/search/mappings.py --- a/src/search/mappings.py +++ b/src/search/mappings.py @@ -21,6 +21,7 @@ PRODUCT_MAPPING = { "properties": { "name": {"type": "text"}, + "tags": {"type": "keyword"}, "price": {"type": "float"} } }
```

EXPECTED OUTPUT — RCA Summary

Field	Expected Value
Root Cause	"tags" field added as keyword type but existing index already mapped it as text
Severity	P2
Contributing Factor 1	No pre-deploy index mapping validation or dry-run against staging index
Contributing Factor 2	Reindex migration not tested against production data volume (2.3M docs)
Key Action Item	Add mapping compatibility check to CI; test migrations on prod-sized dataset

---

All 6 incidents are based on common real-world production failure patterns. Paste the INPUT fields directly into the RCA Drafter form to verify correct output. Compare the generated RCA against the Expected Output table for each incident.